

Step-1: Loading data:

The screenshot shows a SQL IDE window with a dark theme. The title bar indicates the query is 'create raw table' and it has been executed successfully ('OK -- 0 rows -- 22.9ms'). The SQL editor contains a 'CREATE TABLE IF NOT EXISTS' statement for a table named 'superstore_raw'. The table has 22 columns: 'row_id' (INT PRIMARY KEY), 'order_id' (VARCHAR(50)), 'order_date' (VARCHAR(50)), 'ship_date' (VARCHAR(50)), 'ship_mode' (VARCHAR(50)), 'customer_id' (VARCHAR(50)), 'customer_name' (VARCHAR(100)), 'segment' (VARCHAR(50)), 'country' (VARCHAR(100)), 'city' (VARCHAR(100)), 'state' (VARCHAR(100)), 'postal_code' (VARCHAR(20)), 'region' (VARCHAR(50)), 'product_id' (VARCHAR(50)), 'category' (VARCHAR(100)), 'sub_category' (VARCHAR(100)), 'product_name' (VARCHAR(255)), 'sales' (NUMERIC), 'quantity' (INT), 'discount' (NUMERIC), and 'profit' (NUMERIC). The status bar at the bottom shows 'Query executed successfully. No rows returned.'

```
1 CREATE TABLE IF NOT EXISTS superstore_raw (  
2   row_id INT PRIMARY KEY,  
3   order_id VARCHAR(50),  
4   order_date VARCHAR(50),  
5   ship_date VARCHAR(50),  
6   ship_mode VARCHAR(50),  
7   customer_id VARCHAR(50),  
8   customer_name VARCHAR(100),  
9   segment VARCHAR(50),  
10  country VARCHAR(100),  
11  city VARCHAR(100),  
12  state VARCHAR(100),  
13  postal_code VARCHAR(20),  
14  region VARCHAR(50),  
15  product_id VARCHAR(50),  
16  category VARCHAR(100),  
17  sub_category VARCHAR(100),  
18  product_name VARCHAR(255),  
19  sales NUMERIC,  
20  quantity INT,  
21  discount NUMERIC,  
22  profit NUMERIC  
23 );
```

CREATE TABLE IF NOT EXISTS superstore_raw (row_id INT PRIMARY KEY, order_id VARCHAR(50), order_date VARCHAR(50), ship_date VARCHAR(50), ship_mode VARCHAR(50), customer_id VARCHAR(50), customer_name VARCHAR(100), segment VARCHAR(50), country VARCHAR(100), city VARCHAR(100), state VARCHAR(100), postal_code VARCHAR(20), region VARCHAR(50), product_id VARCHAR(50), category VARCHAR(100), sub_category VARCHAR(100), product_name VARCHAR(255), sales NUMERIC, quantity INT, discount NUMERIC, profit NUMERIC); 0 rows affected • 22.9ms

Query executed successfully. No rows returned.

1

The screenshot shows the same SQL IDE window. The title bar indicates the query is 'load csv' and it has been executed successfully ('OK -- 0 rows -- 48.9ms'). The SQL editor contains a 'COPY' statement to load data from a CSV file located at '/tmp/superstore.csv'. The statement specifies the delimiter as a comma, the header row, and the encoding as 'WIN1252'. The status bar at the bottom shows 'Query executed successfully. No rows returned.'

```
1 COPY superstore_raw  
2 FROM '/tmp/superstore.csv'  
3 DELIMITER ',' CSV HEADER ENCODING 'WIN1252';
```

COPY superstore_raw FROM '/tmp/superstore.csv' DELIMITER ',' CSV HEADER ENCODING 'WIN1252'; 9994 rows affected • 48.9ms

Query executed successfully. No rows returned.

2

SQL OK -- 10 rows -- 9.0ms peek data Run Prettify SQL Wiki ↑ ↓ ×

```
1 SELECT * FROM superstore_raw LIMIT 10;
```

10 rows · 9.0ms

	row_id int4	order_id varchar	order_date varchar	ship_date varchar	ship_mode varchar	customer_id varchar	customer_name varchar
1	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Clairmont
2	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Clairmont
3	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darri
4	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	S0-20335	Sean
5	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	S0-20335	Sean
6	6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosi
7	7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosi
8	8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosi
9	9	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosi
10	10	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosi

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 21 9.0ms CSV Excel

3

SQL OK -- 1 row -- 12.2ms row_count Run Prettify SQL Wiki ↑ ↓ ×

```
1 SELECT COUNT(*) as total_rows FROM superstore_raw;
```

1 rows · 12.2ms

	total_rows int8
1	9 994

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 1 COLUMNS: 1 12.2ms CSV Excel

4

Step-2: Creating normalized tables from the raw data:

SQL OK -- 0 rows -- 18.0ms create customers Run Prettify SQL Wiki ↑ ↓ ×

Format SQL

```
1 CREATE TABLE customers AS
2 SELECT DISTINCT
3     customer_id,
4     customer_name,
5     segment
6 FROM
7     superstore_raw;
```

-- customers table with unique customer records CREATE TABLE customers AS SELECT DISTINCT customer_id, customer_name, segment FROM superstore_raw; 793 rows affected • 18.0ms

Query executed successfully. No rows returned.

5

SQL OK -- 0 rows -- 20.9ms create products Run Prettify SQL Wiki ↑ ↓ ×

```
1 -- products table
2
3 CREATE TABLE products AS
4 SELECT DISTINCT
5     product_id,
6     product_name,
7     category,
8     sub_category
9 FROM superstore_raw;
```

-- products table CREATE TABLE products AS SELECT DISTINCT product_id, product_name, category, sub_category FROM superstore_raw; 1894 rows affected • 20.9ms

Query executed successfully. No rows returned.

6

SQL OK -- 0 rows -- 22.4ms create orders Run Prettify SQL Wiki ↑ ↓ ×

Format SQL

```
1 CREATE TABLE orders AS
2 SELECT
3     row_id,
4     order_id,
5     order_date,
6     ship_date,
7     ship_mode,
8     customer_id,
9     country,
10    city,
11    state,
12    postal_code,
13    region,
14    product_id,
15    sales,
16    quantity,
17    discount,
18    profit
19 FROM superstore_raw;
```

-- orders table - this one has all the transactional info CREATE TABLE orders AS SELECT row_id, order_id, order_date, ship_date, ship_mode, customer_id, country, city, state, postal_code, region, product_id, sales, quantity, discount, profit FROM superstore_raw; 9994 rows affected • 22.4ms

Query executed successfully. No rows returned.

7

SQL OK - 3 rows - 4.6ms verify tables Run Prettify SQL Wiki ↑ ↓ ×

```

1  -- quick check on the new tables
2  SELECT 'customers' as tbl, COUNT(*) as cnt FROM customers
3  UNION ALL
4  SELECT 'products', COUNT(*) FROM products
5  UNION ALL
6  SELECT 'orders', COUNT(*) FROM orders;

```

3 rows · 4.6ms

	tbl text	cnt int8
1	customers	793
2	products	1 894
3	orders	9 994

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 3 COLUMNS: 2 4.6ms CSV Excel

8

Step-3: Subqueries:

SQL OK - 15 rows - 14.9ms above avg sales Run Prettify SQL Wiki ↑ ↓ ×

```

2
3  SELECT
4      order_id,
5      customer_id,
6      product_id,
7      sales,
8      profit
9  FROM orders
10 WHERE sales > (SELECT AVG(sales) FROM orders)
11 ORDER BY sales DESC
12 LIMIT 15;

```

15 rows · 14.9ms

	order_id varchar	customer_id varchar	product_id varchar	sales numeric	profit numeric
1	CA-2014-145317	SM-20320	TEC-MA-10002412	22 638.48	-1 811.07
2	CA-2016-118689	TC-20980	TEC-C0-10004722	17 499.95	8 399.97
3	CA-2017-140151	RB-19360	TEC-C0-10004722	13 999.96	6 719.98
4	CA-2017-127180	TA-21385	TEC-C0-10004722	11 199.96	3 919.98
5	CA-2017-166709	HL-15040	TEC-C0-10004722	10 499.97	5 039.98
6	CA-2016-117121	AB-10105	OFF-BI-10000545	9 892.74	4 946.37
7	CA-2014-116904	SC-20095	OFF-BI-10001120	9 449.95	4 630.47
8	US-2016-107440	BS-11365	TEC-MA-10001047	9 099.93	2 365.98
9	CA-2016-158841	SE-20110	TEC-MA-10001127	8 749.95	2 799.98
10	CA-2016-143714	CC-12370	TEC-C0-10004722	8 399.97	1 119.99
11	CA-2014-143917	KL-16645	OFF-SU-10000151	8 187.65	327.50
12	CA-2014-139892	BM-11140	TEC-MA-10000822	8 159.95	-1 359.99

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 15 COLUMNS: 5 14.9ms CSV Excel

9

SQL OK - 1 row - 7.8ms highest sale Run Prettify SQL Wiki ↑ ↓ ×

```

1  --the single highest sales order
2
3  SELECT
4      o.order_id,
5      c.customer_name,
6      p.product_name,
7      o.sales,
8      o.profit
9  FROM orders o
10 JOIN customers c ON o.customer_id = c.customer_id
11 JOIN products p ON o.product_id = p.product_id
12 WHERE o.sales = (SELECT MAX(sales) FROM orders);

```

1 rows · 7.8ms

	order_id	customer_name	product_name	sales	
	varchar	varchar	varchar	numeric	
1	CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferencing Unit	22 638.48	-

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

10

```

2  SELECT
3      o.order_id,
4      p.category,
5      p.product_name,
6      o.sales
7  FROM
8      orders o
9      JOIN products p ON o.product_id = p.product_id
10 WHERE
11     o.sales > (
12         SELECT
13             AVG(oo.sales)
14         FROM
15             orders oo
16             JOIN products pp ON oo.product_id = pp.product_id
17         WHERE
18             pp.category = p.category
19     )
20 ORDER BY
21     o.sales DESC
22 LIMIT
23     20;

```

20 rows · 17.55s

order_id	category	product_name	sales
varchar	varchar	varchar	numeric
914-145317	Technology	Cisco TelePresence System EX90 Videoconferencing Unit	22 638.48
916-118689	Technology	Canon imageCLASS 2200 Advanced Copier	17 499.00
917-140151	Technology	Canon imageCLASS 2200 Advanced Copier	13 999.00
917-127180	Technology	Canon imageCLASS 2200 Advanced Copier	11 199.00
917-166709	Technology	Canon imageCLASS 2200 Advanced Copier	10 499.00
916-117121	Office Supplies	GBC Ibimaster 500 Manual ProClick Binding System	9 892.00
914-116904	Office Supplies	Ibico EPK-21 Electric Binding System	9 449.00
916-107440	Technology	3D Systems Cube Printer, 2nd Generation, Magenta	9 099.00
916-159841	Technology	HP DesignJet T520 Inkjet Large Format Printer, 24" Color	9 740.00

11.orders that beat the average sales of their own category

```

1  --customers who have placed more orders than the average customer
2  SELECT
3      customer_id,
4      order_count
5  FROM
6      (
7          SELECT
8              customer_id,
9              COUNT(DISTINCT order_id) AS order_count
10             FROM
11                 orders
12             GROUP BY
13                 customer_id
14         ) sub
15  WHERE
16      order_count > (
17          SELECT
18              AVG(order_count)
19          FROM
20              (
21                  SELECT
22                      COUNT(DISTINCT order_id) AS order_count
23                  FROM
24                      orders
25                  GROUP BY
26                      customer_id
27              ) avg_sub
28          )
29  ORDER BY
30      order_count DESC;

```

357 rows · 91.0ms

	customer_id varchar	order_count int8
1	EP-13915	17
2	EA-14035	13
3	PG-18820	13
4	SH-19975	13

12. customers who have placed more orders than the average customer

SQL OK -- 5 rows -- 18.4ms top5_products sl Run Prettify SQL Wiki ↑ ↓ ×

```

1  --top 5 products by total sales using subquery in FROM
2
3  SELECT
4      p.product_name,
5      p.category,
6      prod_sales.total_sales
7  FROM (
8      SELECT product_id, ROUND(SUM(sales),2) as total_sales
9      FROM orders
10     GROUP BY product_id
11     ORDER BY total_sales DESC
12     LIMIT 5
13 ) prod_sales
14 JOIN products p ON prod_sales.product_id = p.product_id
15 ORDER BY prod_sales.total_sales DESC;

```

5 rows · 18.4ms

	product_name varchar	category varchar	total_sales numeric
1	Canon imageCLASS 2200 Advanced Copier	Technology	61 599.82
2	Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	Office Supplies	27 453.38
3	Cisco TelePresence System EX90 Videoconferencing Unit	Technology	22 638.48
4	HON 5400 Series Task Chairs for Big and Tall	Furniture	21 870.58
5	GBC DocuBind TL300 Electric Binding System	Office Supplies	19 823.48

Step-4: CTEs:

```

2  WITH
3  cust_met AS (
4  SELECT
5      o.customer_id,
6      c.customer_name,
7      c.segment,
8      COUNT(DISTINCT o.order_id) AS total_orders,
9      ROUND(SUM(o.sales), 2) AS total_sales,
10     ROUND(SUM(o.profit), 2) AS total_profit,
11     ROUND(AVG(o.sales), 2) AS avg_order_value
12 FROM
13     orders o
14 JOIN customers c ON o.customer_id = c.customer_id
15 GROUP BY
16     o.customer_id,
17     c.customer_name,
18     c.segment
19 )
20 SELECT
21     *
22 FROM
23     cust_met
24 ORDER BY
25     total_sales DESC
26 LIMIT
27     15;

```

15 rows - 56.0ms

customer_id	customer_name	segment	total_orders	total_sales	total_profit	avg_order_value
0320	Sean Miller	Home Office	5	25 043.05	-1 980.74	1 669.67
0980	Tamara Chand	Corporate	5	19 052.22	8 981.32	1 580.44
9360	Raymond Buch	Consumer	6	15 117.34	6 976.10	839.59
1385	Tom Ashbrook	Home Office	4	14 595.62	4 703.79	1 459.41
0105	Adrian Barton	Consumer	10	14 473.57	5 444.81	723.67
6645	Ken Lonsdale	Consumer	12	14 175.23	806.86	488.12

14. customer-level sales summary using CTE

Step-5: Window functions:

```

1  SELECT
2  customer_id,
3  order_id,
4  sales,
5  ROW_NUMBER() OVER (
6      PARTITION BY
7          customer_id
8      ORDER BY
9          sales DESC
10 ) AS row_num
11 FROM
12     orders
13 LIMIT
14     20;

```

20 rows - 23.8ms

	customer_id	order_id	sales	row_num
1	AA-10315	CA-2016-103982	3 930.07	1
2	AA-10315	CA-2014-128055	673.56	2
3	AA-10315	CA-2016-103982	431.97	3
4	AA-10315	CA-2017-147039	362.94	4
5	AA-10315	CA-2014-128055	52.98	5
6	AA-10315	CA-2016-103982	41.72	6
7	AA-10315	CA-2015-121391	26.96	7
8	AA-10315	CA-2014-138100	14.94	8
9	AA-10315	CA-2014-138100	14.56	9
10	AA-10315	CA-2017-147039	11.54	10
11	AA-10315	CA-2016-103982	2.30	11
12	AA-10375	CA-2016-131065	499.98	1

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

```

3 SELECT * FROM (
4     SELECT
5         o.customer_id,
6         c.customer_name,
7         o.order_id,
8         o.sales,
9         ROW_NUMBER() OVER(
10            PARTITION BY o.customer_id
11            ORDER BY o.sales DESC
12        ) as row_num
13     FROM orders o
14     JOIN customers c ON o.customer_id = c.customer_id
15 ) ranked
16 WHERE row_num = 1
17 ORDER BY sales DESC
18 LIMIT 10;

```

10 rows · 28.6ms

☒	customer_id varchar	customer_name varchar	order_id varchar	sales numeric	row_num int8
1	SM-20320	Sean Miller	CA-2014-145317	22 638.48	1
2	TC-20980	Tamara Chand	CA-2016-118689	17 499.95	1
3	RB-19360	Raymond Buch	CA-2017-140151	13 999.96	1
4	TA-21385	Tom Ashbrook	CA-2017-127180	11 199.96	1
5	HL-15040	Hunter Lopez	CA-2017-166709	10 499.97	1
6	AB-10105	Adrian Barton	CA-2016-117121	9 892.74	1
7	SC-20095	Sanjit Chand	CA-2014-116904	9 449.95	1
8	BS-11365	Bill Shonely	US-2016-107440	9 099.93	1
9	SE-20110	Sanjit Engle	CA-2016-158841	8 749.95	1
10	CC-12370	Christopher Conant	CA-2016-143714	8 399.97	1

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 10 COLUMNS: 5 28.6ms

CSV

Excel

17.get each customer's top order (their highest sale)


```

2 WITH
3   cust_totals AS (
4     SELECT
5       customer_id,
6       ROUND(SUM(sales), 2) AS total_sales
7     FROM
8       orders
9     GROUP BY
10      customer_id
11   )
12 SELECT
13   customer_id,
14   total_sales,
15   RANK() OVER (
16     ORDER BY
17       total_sales DESC
18   ) AS rank_num,
19   DENSE_RANK() OVER (
20     ORDER BY
21       total_sales DESC
22   ) AS dense_rank_num,
23   ROW_NUMBER() OVER (
24     ORDER BY
25       total_sales DESC
26   ) AS row_num
27 FROM
28   cust_totals
29 LIMIT
30   20;

```

20 rows · 16.2ms

	customer_id varchar	total_sales numeric	rank_num int8	dense_rank_num int8	row_num int8
1	SM-20320	25 043.05	1	1	1
2	TC-20980	19 052.22	2	2	2
3	RB-19360	15 117.34	3	3	3
4	TA-21385	14 595.62	4	4	4

18.rank vs dense_rank vs row_num comparison on customer total sales

```

1  -- Rank products within each category by total sales
2
3  WITH product_totals AS (
4      SELECT
5          p.product_id,
6          p.product_name,
7          p.category,
8          ROUND(SUM(o.sales),2) as total_sales
9      FROM orders o
10     JOIN products p ON o.product_id = p.product_id
11     GROUP BY p.product_id, p.product_name, p.category
12 )
13 SELECT
14     category,
15     product_name,
16     total_sales,
17     DENSE_RANK() OVER(
18         PARTITION BY category
19         ORDER BY total_sales DESC
20     ) as category_rank
21 FROM product_totals
22 ORDER BY category, category_rank
23 LIMIT 25;

```

25 rows · 32.3ms

	total_sales numeric	category_rank int8
Task Chairs for Big and Tall	21 870.58	1
s Royal Lawyers Bookcase, Royale Cherry Finish	15 610.97	2
gular Conference Table Tops	12 995.29	3
ecutive Leather Low-Back Tilter	12 975.38	4
Hills Library, Woodland Oak Finish	12 921.64	5
ecutive Suite Bookcases	12 921.64	5
ling Chair	11 572.78	6
ric Upholstered Stacking Chairs, Rounded Back	10 637.53	7

19.rank products within each category by total sales

```

3 SELECT
4     o.customer_id,
5     c.customer_name,
6     o.order_id,
7     o.order_date,
8     o.sales,
9     SUM(o.sales) OVER(
10        PARTITION BY o.customer_id
11        ORDER BY TO_DATE(o.order_date, 'MM/DD/YYYY')
12        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
13    ) as running_total
14 FROM orders o
15 JOIN customers c ON o.customer_id = c.customer_id
16 ORDER BY o.customer_id, TO_DATE(o.order_date, 'MM/DD/YYYY')
17 LIMIT 25;

```

25 rows · 22.4ms

☒	customer_id	customer_name	order_id	order_date	sales	running_total
	varchar	varchar	varchar	varchar	numeric	numeric
1	AA-10315	Alex Avila	CA-2014-128055	3/31/2014	52.98	52.98
2	AA-10315	Alex Avila	CA-2014-128055	3/31/2014	673.56	726.54
3	AA-10315	Alex Avila	CA-2014-138100	9/15/2014	14.56	741.10
4	AA-10315	Alex Avila	CA-2014-138100	9/15/2014	14.94	756.04
5	AA-10315	Alex Avila	CA-2015-121391	10/4/2015	26.96	783.00
6	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	3 930.07	4 713.08
7	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	431.97	5 145.05
8	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	41.72	5 186.77
9	AA-10315	Alex Avila	CA-2016-103982	3/3/2016	2.30	5 189.08
10	AA-10315	Alex Avila	CA-2017-147039	6/29/2017	362.94	5 552.02
11	AA-10315	Alex Avila	CA-2017-147039	6/29/2017	11.54	5 563.56
12	AA-10375	Allen Arnold	CA-2014-158064	4/21/2014	16.52	16.52

20. running total of sales per customer (ordered by order date)

Step-6: Combining join, cte, windows functions:

```

3 WITH cust_agg AS (
4     SELECT
5         o.customer_id,
6         c.customer_name,
7         c.segment,
8         COUNT(DISTINCT o.order_id) as num_orders,
9         ROUND(SUM(o.sales), 2) as total_sales,
10        ROUND(SUM(o.profit), 2) as total_profit,
11        ROUND(AVG(o.sales), 2) as avg_sale
12    FROM orders o
13    JOIN customers c ON o.customer_id = c.customer_id
14    GROUP BY o.customer_id, c.customer_name, c.segment
15 )
16 SELECT
17     customer_name,
18     segment,
19     num_orders,
20     total_sales,
21     total_profit,
22     avg_sale,
23     RANK() OVER(ORDER BY total_sales DESC) as overall_rank,
24     RANK() OVER(PARTITION BY segment ORDER BY total_sales DESC) as segment_rank
25 FROM cust_agg
26 ORDER BY overall_rank
27 LIMIT 20;

```

20 rows · 38.2ms

☒	customer_name  	segment  	num_orders  	total_sales  	total_profit  	avg_sale  
	varchar	varchar	int8	numeric	numeric	numeric
1	Sean Miller	Home Office	5	25 043.05	-1 980.74	1 669.54
2	Tamara Chand	Corporate	5	19 052.22	8 981.32	1 587.68
3	Raymond Buch	Consumer	6	15 117.34	6 976.10	839.85
4	Tom Ashbrook	Home Office	4	14 595.62	4 703.79	1 459.56
5	Adrian Barton	Consumer	10	14 473.57	5 444.81	723.68
6	Ken Lonsdale	Consumer	12	14 175.23	806.86	488.80
7	Sanjit Chand	Consumer	9	14 142.33	5 757.41	642.83
8	Hunter Lopez	Consumer	6	12 873.30	5 622.43	1 170.30
9	Sanjit Engle	Consumer	11	12 209.44	2 650.68	642.60
10	Christopher Conant	Consumer	5	12 129.07	2 177.05	1 102.64
11	Todd Sumrall	Corporate	6	11 891.75	2 371.71	792.78
12	Greg Tran	Consumer	11	11 820.12	2 163.43	407.59

21.full customer ranking report with segment-level ranks

```

1  --top 3 customers per segment
2
3  WITH cust_sales AS (
4      SELECT
5          o.customer_id,
6          c.customer_name,
7          c.segment,
8          ROUND(SUM(o.sales),2) as total_sales
9      FROM orders o
10     JOIN customers c ON o.customer_id = c.customer_id
11     GROUP BY o.customer_id, c.customer_name, c.segment
12 ),
13 ranked AS (
14     SELECT
15         *,
16         ROW_NUMBER() OVER(PARTITION BY segment ORDER BY total_sales DESC) as seg_rank
17     FROM cust_sales
18 )
19 SELECT
20     segment,
21     customer_name,
22     total_sales,
23     seg_rank
24 FROM ranked
25 WHERE seg_rank <= 3
26 ORDER BY segment, seg_rank;

```

9 rows · 26.4ms

☒	segment varchar	customer_name varchar	total_sales numeric	seg_rank int8
1	Consumer	Raymond Buch	15 117.34	1
2	Consumer	Adrian Barton	14 473.57	2
3	Consumer	Ken Lonsdale	14 175.23	3
4	Corporate	Tamara Chand	19 052.22	1
5	Corporate	Todd Sumrall	11 891.75	2
6	Corporate	Bill Shonely	10 501.65	3
7	Home Office	Sean Miller	25 043.05	1
8	Home Office	Tom Ashbrook	14 595.62	2
9	Home Office	Maria Etezadi	10 663.73	3

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 9 COLUMNS: 4 26.4ms

CSV

Excel

```

1  --each customer's best selling product category
2
3  WITH cust_cat_sales AS (
4      SELECT
5          o.customer_id,
6          c.customer_name,
7          p.category,
8          ROUND(SUM(o.sales),2) as cat_sales
9      FROM orders o
10     JOIN customers c ON o.customer_id = c.customer_id
11     JOIN products p ON o.product_id = p.product_id
12     GROUP BY o.customer_id, c.customer_name, p.category
13 ),
14 ranked_cats AS (
15     SELECT *,
16         ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY cat_sales DESC) as row_num
17     FROM cust_cat_sales
18 )
19 SELECT
20     customer_name,
21     category as top_category,
22     cat_sales
23 FROM ranked_cats
24 WHERE row_num = 1
25 ORDER BY cat_sales DESC
26 LIMIT 15;

```

15 rows · 36.1ms

☒	customer_name varchar	top_category varchar	cat_sales numeric
1	Sean Miller	Technology	23 481.51
2	Tamara Chand	Technology	17 997.95
3	Raymond Buch	Technology	14 265.42
4	Tom Ashbrook	Technology	13 709.96
5	Sanjit Chand	Office Supplies	12 081.26
6	Hunter Lopez	Technology	11 640.81
7	Adrian Barton	Office Supplies	11 489.26
8	Christopher Conant	Technology	11 322.32
9	Greg Tran	Furniture	10 227.10
10	Ken Lonsdale	Office Supplies	9 708.67
11	Sanjit Engle	Technology	9 351.99
12	Bill Shonely	Technology	9 106.83

23

Step-7: some business queries:

2	
3	WITH cust_summary AS (
4	SELECT
5	o.customer_id,
6	c.customer_name,
7	c.segment,
8	ROUND(SUM(o.sales),2) as total_sales,
9	ROUND(SUM(o.profit),2) as total_profit,
10	COUNT(DISTINCT o.order_id) as order_count
11	FROM orders o
12	JOIN customers c ON o.customer_id = c.customer_id
13	GROUP BY o.customer_id, c.customer_name, c.segment
14	)
15	SELECT *
16	FROM cust_summary
17	ORDER BY total_sales DESC
18	LIMIT 10;
10 rows · 34.3ms	
☒	customer_id varchar
	customer_name varchar
	segment varchar
	total_sales numeric
	total_profit numeric
	order_count int8
1	SM-20320 Sean Miller Home Office 25 043.05 -1 980.74 5
2	TC-20980 Tamara Chand Corporate 19 052.22 8 981.32 5
3	RB-19360 Raymond Buch Consumer 15 117.34 6 976.10 6
4	TA-21385 Tom Ashbrook Home Office 14 595.62 4 703.79 4
5	AB-10105 Adrian Barton Consumer 14 473.57 5 444.81 10
6	KL-16645 Ken Lonsdale Consumer 14 175.23 806.86 12
7	SC-20095 Sanjit Chand Consumer 14 142.33 5 757.41 9
8	HL-15040 Hunter Lopez Consumer 12 873.30 5 622.43 6
9	SE-20110 Sanjit Engle Consumer 12 209.44 2 650.68 11
10	CC-12370 Christopher Conant Consumer 12 129.07 2 177.05 5
Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)	

24. top 10 customers by total sales

```

2
3 WITH cust_summary AS (
4     SELECT
5         o.customer_id,
6         c.customer_name,
7         c.segment,
8         ROUND(SUM(o.sales),2) as total_sales,
9         ROUND(SUM(o.profit),2) as total_profit,
10        COUNT(DISTINCT o.order_id) as order_count
11    FROM orders o
12    JOIN customers c ON o.customer_id = c.customer_id
13    GROUP BY o.customer_id, c.customer_name, c.segment
14 )
15 SELECT *
16 FROM cust_summary
17 ORDER BY total_sales ASC
18 LIMIT 10;

```

10 rows · 53.3ms

☒	customer_id	customer_name	segment	total_sales	total_profit	order_count
	varchar	varchar	varchar	numeric	numeric	int8
1	TS-21085	Thais Sissman	Consumer	4.83	-3.32	2
2	LD-16855	Lela Donovan	Corporate	5.30	0.46	1
3	CJ-11875	Carl Jackson	Corporate	16.52	1.65	1
4	MG-18205	Mitch Gastineau	Corporate	16.74	-1.25	1
5	RS-19870	Roy Skaria	Home Office	22.33	9.58	2
6	SG-20890	Susan Gilcrest	Corporate	47.95	-3.71	3
7	RE-19405	Ricardo Emerson	Consumer	48.36	6.05	1
8	LB-16735	Larry Blacks	Consumer	50.19	18.65	3
9	AS-10135	Adrian Shami	Home Office	58.82	21.85	2
10	JC-15340	Jasper Cacioppo	Consumer	71.26	-0.36	4

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

25. bottom 10 customers

<pre> 2 3 WITH cust_orders AS (4 SELECT 5 o.customer_id, 6 c.customer_name, 7 c.segment, 8 COUNT(DISTINCT o.order_id) as order_count, 9 ROUND(SUM(o.sales),2) as total_sales 10 FROM orders o 11 JOIN customers c ON o.customer_id = c.customer_id 12 GROUP BY o.customer_id, c.customer_name, c.segment 13) 14 SELECT * 15 FROM cust_orders 16 WHERE order_count = 1 17 ORDER BY total_sales DESC; </pre>					
12 rows · 33.3ms					
	customer_id varchar	customer_name varchar	segment varchar	order_count int8	total_sales numeric
1	JC-15385	Jenna Caffey	Consumer	1	1 058.11
2	SM-20905	Susan MacKendrick	Consumer	1	1 043.04
3	TC-21145	Theresa Coyne	Corporate	1	1 038.26
4	JR-15700	Jocasta Rupert	Consumer	1	863.88
5	PH-18790	Patricia Hirasaki	Home Office	1	729.65
6	AO-10810	Anthony O'Donnell	Corporate	1	161.28
7	RM-19750	Roland Murray	Consumer	1	98.35
8	AR-10570	Anemone Ratner	Consumer	1	88.15
9	RE-19405	Ricardo Emerson	Consumer	1	48.36
10	MG-18205	Mitch Gastineau	Corporate	1	16.74
11	CJ-11875	Carl Jackson	Corporate	1	16.52
12	LD-16855	Lela Donovan	Corporate	1	5.30

26. customers who paid for single order

26. customers who placed a single order

SQL OK -- 2 rows - 17.6ms customer_distrib Run Prettify SQL Wiki ↑ ×

```

1  --customer distribution summary,i.e., how many are above avg vs below avg
2  WITH
3      cust_totals AS (
4          SELECT
5              customer_id,
6              SUM(sales) AS total_sales
7          FROM
8              orders
9          GROUP BY
10             customer_id
11      ),
12      avg_calc AS (
13          SELECT
14              AVG(total_sales) AS avg_sales
15          FROM
16              cust_totals
17      )
18  SELECT
19      CASE
20          WHEN ct.total_sales > ac.avg_sales THEN 'Above Average'
21          ELSE 'Below Average'
22      END AS cust_grp,
23      COUNT(*) AS num_customers,
24      ROUND(AVG(ct.total_sales), 2) AS group_avg_sales,
25      ROUND(MIN(ct.total_sales), 2) AS min_sales,
26      ROUND(MAX(ct.total_sales), 2) AS max_sales
27  FROM
28      cust_totals ct
29      CROSS JOIN avg_calc ac
30  GROUP BY
31      cust_grp
32  ORDER BY
33      group_avg_sales DESC;

```

2 rows - 17.6ms

	cust_grp	num_customers	group_avg_sales	min_sales	max_sales
1	Above Average	294	5 362.37	2 900.03	25 043.05
2	Below Average	499	1 444.22	4.83	2 893.46

Enter a WHERE clause to filter results (e.g. status = 'active' AND id > 10)

ROWS: 2 COLUMNS: 5 17.6ms CSV Excel